arXiv:2402.19155v1 [cs.LG] 29 Feb 2024

Beyond Language Models: Byte Models are Digital World Simulators

Shangda Wu [1 2] * **Xu Tan** [1] * **Zili Wang** [3] **Rui Wang** [1] **Xiaobing Li** [2] **Maosong Sun** [2 4]
https://byte-gpt.github.io

Abstract

Traditional deep learning often overlooks bytes, the basic units of the digital world, where all forms of information and operations are encoded and manipulated in binary format. Inspired by the success of next token prediction in natural language processing, we introduce bGPT, a model with next byte prediction to simulate the digital world. bGPT matches specialized models in performance across various modalities, including text, audio, and images, and offers new possibilities for predicting, simulating, and diagnosing algorithm or hardware behaviour. It has almost flawlessly replicated the process of converting symbolic music data, achieving a low error rate of 0.0011 bits per byte in converting ABC notation to MIDI format. In addition, bGPT demonstrates exceptional capabilities in simulating CPU behaviour, with an accuracy exceeding 99.99% in executing various operations. Leveraging next byte prediction, models like bGPT can directly learn from vast binary data, effectively simulating the intricate patterns of the digital world.

1. Introduction

Deep learning research has focused on digital media files that are easily interpretable by humans, such as text, audio, and images (Oord et al., 2016; He et al., 2016; Peters et al., 2018), due to their direct relevance to human communication and understanding. Text, in particular, plays a central role in conveying human intelligence and has led to the emergence of Language Models (LMs) (Radford et al., 2018; 2019; Brown et al., 2020; Bubeck et al., 2023; Touvron et al., 2023a;b). The fundamental principle of LM involves tokenizing text (Sennrich et al., 2016; Wu et al., 2016; Kudo, 2018; Kudo & Richardson, 2018) and predicting the next token in a sequence, enabling them to comprehend human language and intelligence. Recent advancements further extend tokenization to various modalities beyond text (Dosovitskiy et al., 2021; Défossez et al., 2022), empowering LMs to attain a more holistic understanding of the real world and emulate human intelligence (Wang et al., 2023; Borsos et al., 2023; Liu et al., 2023; Li et al., 2023).

These deep learning models, however, predominantly operate within the realm of media data, overlooking the omnipresent native binary data in the digital world. Bytes are the foundation of all digital data, devices, and software, from computer processors to operating systems in everyday electronics. Therefore, training models for next byte prediction can potentially lead to a paradigm shift in deep learning, allowing them to truly understand and simulate all activities in the digital world. This has practical benefits not only in conventional areas, but also in some underexplored areas such as boosting cybersecurity (Raff et al., 2018), improving computer diagnostics (Guo et al., 2019), optimizing data compression (Delétang et al., 2023), and even advancing complex tasks like reverse-engineering the source code of that software from its binary representation.

In this paper, we introduce bGPT, a model designed for binary data processing and digital world modelling by next byte prediction. The digital world includes not only digital media files, traditionally the focus of deep learning models, but also extends to the intricate realm of digital systems, ranging from hardware architectures to complex algorithms. bGPT transcends traditional deep learning boundaries by directly interpreting and manipulating binary data, enabling a more intrinsic and holistic understanding of the digital world. Its advantages are two-fold: 1) **Interpreting Digital System**: By training on byte sequences, bGPT can learn the patterns of digital systems, enabling it to predict, simulate, and diagnose algorithm or hardware behaviour. This ability allows for the reconstruction of complex systems from binary data. 2) **Unified Modelling**: bGPT integrates various data types into a single framework, treating everything as a byte sequence. This simplifies modelling and allows for easy integration of various data sources.

Our experiments include two main areas: 1) well-studied tasks like generative modelling and classification on digital media data (e.g., text, audio, and images); and 2) relatively

*Equal contribution [1]Microsoft Research Asia [2]Central Conservatory of Music, China [3]Independent researcher [4]Tsinghua University, China. Correspondence to: Maosong Sun <tsinghua.edu.cn>.

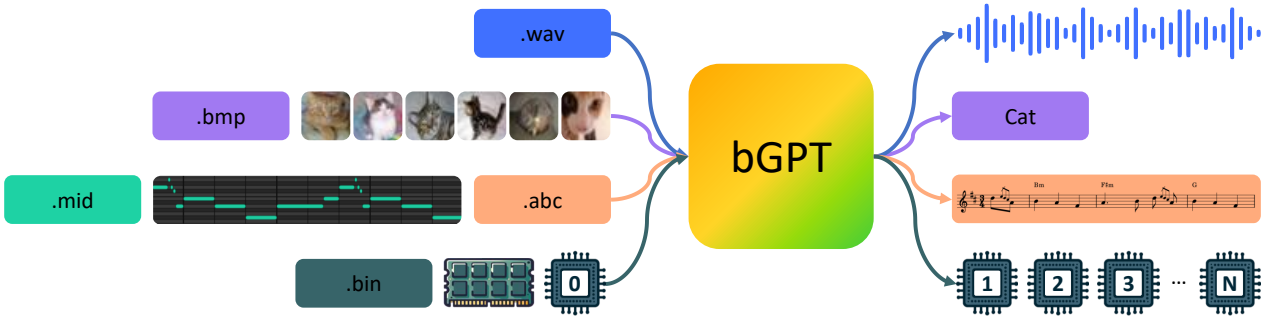


Figure 1. The bGPT framework simulates digital systems through native binary data, and integrates diverse data types into a single model, treating everything as a byte sequence. This approach simplifies integration and expands application possibilities in the digital world.

underexplored tasks intrinsic to binary-native operations, including data conversion and CPU state modelling, which represent algorithm and hardware simulation, respectively. The study of byte models not only marks a significant step towards holistic and unified deep learning, but also offers a new perspective on modelling the digital world.

2. Background

2.1. Language Models

LMs, from earlier LSTM-based (Hochreiter & Schmidhuber, 1997) to recent Transformer-based models (Vaswani et al., 2017), are crucial for both understanding and generating human language while simulating human intelligence. Text tokenization plays a fundamental role in these models, as it involves breaking down text into smaller units, such as words or subwords (Sennrich et al., 2016; Wu et al., 2016; Kudo, 2018; Kudo & Richardson, 2018), which serve as the input for the model. The introduction of Generative Pre-trained Transformer (GPT) models (Radford et al., 2018; 2019; Brown et al., 2020) represents a significant advancement in this field. GPT models are pre-trained through self-supervised learning, particularly via next token prediction on extensive text data. This training technique, next token prediction, teaches the model to predict the most likely next token in a sequence, enabling it to capture the structure and semantics behind languages.

Next token prediction has extended its influence to various data types. In audio processing, models like AudioPaLM (Rubenstein et al., 2023) merge text and speech, enabling speech-to-speech translation and advanced speech recognition. MusicGen (Copet et al., 2023) excels in conditional music generation by modelling multiple parallel streams of acoustic tokens extracted by EnCodec (Défossez et al., 2022). In image processing, iGPT (Chen et al., 2020) applies Transformer models to predict the next pixel in an image, while several vision-language models (Liu et al., 2023; Zhu et al., 2023; Li et al., 2023) have emerged to bridge the gap between textual and visual data. In biochemical sequences, Tranception (Notin et al., 2022) leverages autoregressive transformers and retrieval to predict protein fitness, while ProtGPT2 (Ferruz et al., 2022) generates protein sequences with natural amino acid propensities. HyenaDNA (Nguyen et al., 2023) extends context lengths in genomic modelling, enabling long-range sequence understanding.

Next token prediction has empowered LMs to grasp the intricacies of human intelligence and the world. Expanding these techniques to binary data via next byte prediction could further enhance their versatility in handling digital information and simulating the digital world.

2.2. Byte Models

While binary data lacks the inherent structure and semantics of human-interpretable data like text, recent research efforts are exploring its modelling and information extraction, opening up new possibilities for byte models.

By modelling native binary data, systems like MalConv (Raff et al., 2018) and DeepVSA (Guo et al., 2019) have emerged as potent tools for malware detection and program analysis. MalConv employs Convolutional Neural Networks (CNNs) (LeCun et al., 1989) to analyze raw byte sequences in executable files, while DeepVSA enhances memory alias analysis within the context of value set analysis for post-mortem program analysis. Additionally, the concept of language models compressing byte sequences (Delétang et al., 2023) introduces a novel perspective on how large pre-trained models (Hoffmann et al., 2022) can be utilized.

Several studies have validated the utility of byte-level encoding for language tasks. For instance, Byte-level Byte Pair Encoding (BBPE) has been used to enhance multilingual model pre-training (Wei et al., 2021) and has also shown promise in machine translation (Wang et al., 2020), striking a balance between processing efficiency and linguistic breadth. ByT5 (Xue et al., 2022) builds on this by using standard Transformer models for byte sequences, promoting a token-free encoding method that improves noise robustness and spelling sensitivity in multilingual scenarios.

Byte encoding has also been applied to other human-interpretable data, allowing models to work with binary representations of text, images, and diverse data types in a universal framework. For example, ByteFormer (Horton et al., 2023) directly handles raw byte sequences converted from images and audio while maintaining versatility and privacy. MegaByte (Yu et al., 2023), on the other hand, has been tested and proven to excel in modelling long byte sequences across various modalities. Inspired by MegaByte (Yu et al., 2023), MambaByte (Wang et al., 2024) leverages the Mamba network structure (Gu & Dao, 2023) to excel in byte-level language modelling and even outperforms LMs based on subword tokenization.

Despite progress, current research often neglects native binary data, focusing on narrow tasks and overlooking the broader potential of byte models in digital world simulation. To address these issues, we employed bGPT to model native binary data and conducted comprehensive evaluations across various tasks. This approach provides a holistic assessment of byte models in various applications, offering insights into the potential of digital world modelling.

3. Methodology

In this section, we introduce bGPT, a model optimized for modelling digital data at the byte level. We start by presenting its hierarchical Transformer framework, which segments byte sequences into patches to manage computational efficiency. Subsequently, we present the training objectives of bGPT, including generative modelling and classification.

3.1. Model Architecture

Working with digital data at the byte level not only allows models to learn digital system patterns but also offers a unified approach for incorporating various data types into a single framework. However, the high granularity of bytes results in long sequences, substantially raising computational costs. This issue is more pronounced in Transformer-based models due to quadratic self-attention scaling, restricting their efficiency and scalability for handling binary data.

To address computational limitations in Transformer-based models with long byte sequences, inspired by previous work (Wu et al., 2023b; Yu et al., 2023; Wu et al., 2023a; Yang et al., 2023), bGPT adapts a hierarchical Transformer architecture and segments a sequence of bytes $B = \{b_1, b_2, ..., b_T\}$ of length T into a sequence of patches $\mathcal{P}$, where each patch contains exactly S bytes:

$$\mathcal{P} = [P_1, P_2, \ldots, P_N], \tag{1}$$

where $N = \lceil \frac{T}{S} \rceil$ is the number of patches, and $P_i = [b_{(i-1)S+1}, \ldots, b_{iS}]$ for $1 \leq i \leq N$. If $T \mod S \neq 0$, the last patch P_N is defined as:

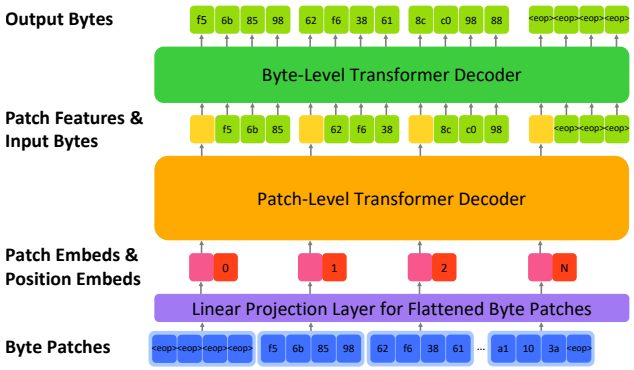


Figure 2. bGPT segments byte sequences into patches, predicts next patch features with a patch-level decoder, and reconstructs bytes within patches using these features with a byte-level decoder.

$$P_N = [b_{(N-1)S+1}, \ldots, b_T, \underbrace{e, \ldots, e}_{S-(T \mod S)}], \tag{2}$$

where e represents the <eop> (end-of-patch) token used to pad the last patch P_N to the size S. By segmenting byte sequences into more manageable patches, bGPT balances the need for byte-level sequence modelling with computational efficiency.

As illustrated in Fig. 2, bGPT consists of three main components: a linear projection layer, a patch-level decoder, and a byte-level decoder. bGPT processes the patch sequence $\mathcal{P}$ through its components as follows:

Linear Projection Layer: Each patch P_i from $\mathcal{P}$ is viewed as a matrix of size $S \times 257$, where S is the patch size, and each byte is one-hot encoded into a 257D vector, including all 256 byte values and an <eop> token. These patches are then flattened into one-dimensional vectors, where the rows in the matrix are concatenated sequentially. The linear projection layer then maps each flattened vector into a dense vector E_i of a hidden size H. For each patch, the operation can be formulated as:

$$E_i = \text{Flatten}(P_i) \cdot W_{\text{linear}}, \quad 1 \leq i \leq N, \tag{3}$$

where W_{linear} is the weight matrix of the linear projection layer with a shape of $(S \times 257, H)$. This dense embedding enables more efficient processing of the byte sequence by reducing the dimensionality while preserving the essential information contained in each patch.

Patch-Level Decoder: This decoder takes the sequence of embedded patches $\mathcal{E} = \{E_1, E_2, \ldots, E_N\}$ and processes it to autoregressively predict the features of the subsequent patch, effectively learning the structure of the data:

$$\hat{E}_i = \text{Decoder}_{\text{patch}}(\mathcal{E}_{<i} \oplus \mathcal{X}_{<i}), \tag{4}$$

where $\mathcal{E}_{<i}$ denotes the sequence of patch embeddings before the i-th patch, and $\mathcal{X}_{<i}$ represents the corresponding

positional embeddings. The $\oplus$ symbol denotes the element-wise addition of these two sequences. The output, $\hat{E}_i$, is the predicted feature for the i-th patch.

Byte-Level Decoder: It takes the predicted feature $\hat{E}_i$ of an individual patch and autoregressively reconstructs the sequence of bytes within that patch. The process is independent for each patch and operates by conditioning on the feature representation $\hat{E}_i$ of the current patch:

$$\hat{b}_{i,j} = \text{Decoder}_{\text{byte}}(\hat{E}_i, b_{i,<j}), \quad 1 \leq j \leq S, \tag{5}$$

where $\hat{b}_{i,j}$ is the predicted byte at position j in the i-th patch, and $b_{i,<j}$ represents all preceding bytes in the current patch.

3.2. Training Objectives

The training for bGPT primarily revolves around generative modelling through next byte prediction as its core focus, playing a pivotal role in predicting and generating bytes.

Generative Modelling: It aims to predict the next byte b_{i+1} in a sequence based on preceding bytes $\{b_1, b_2, ..., b_i\}$ without explicit guidance. For a byte sequence $B = \{b_1, b_2, ..., b_T\}$ of length T, the objective is minimizing the negative log-likelihood of the next byte prediction across the sequence, defined as:

$$\mathcal{L}_{\text{GEN}}(\theta) = -\sum_{i=1}^{T-1} \log p(b_{i+1}|b_1, b_2, ..., b_i; \theta), \tag{6}$$

where θ represents the model parameters, and $p(\cdot)$ denotes the predicted probability distribution over possible next bytes. The loss function $\mathcal{L}_{\text{GEN}}(\cdot)$ in generative modelling encourages the model to understand the sequential dependencies in data at the byte level.

After being initially trained on generative modelling via next byte prediction, bGPT is further adapted to classification tasks with a new training objective.

Classification: Upon the foundation of generative modelling through next byte prediction, bGPT is further trained on labelled datasets, where it predicts categories from byte sequences. This involves extracting a global feature from the byte sequence, which is then processed by a classification head. The goal is to minimize the classification loss $\mathcal{L}_{\text{CLF}}$, formulated as:

$$\mathcal{L}_{\text{CLF}}(\theta) = -\sum_{k=1}^{K} y_k \log p(y_k|B; \theta), \tag{7}$$

where y_k is the boolean label for the k-th category, indicating whether the byte sequence belongs (true) or does not belong (false) to that category. K is the total number of categories, and $p(y_k|B; \theta)$ is the predicted probability of category k given the byte sequence B.

4. Applications

Byte models such as bGPT excel at understanding binary data, and have proficiency in digital media file processing (e.g., text, audio, and images) and simulating algorithms and hardware operations for in-depth software and digital system analysis. This section introduces the selected applications and corresponding experimental settings for evaluating the proposed bGPT.

4.1. Digital Media Processing

The field of deep learning is steadily advancing its proficiency in both the generation and classification of diverse forms of media, including text, audio, and images (Devlin et al., 2018; Ao et al., 2022; Ramesh et al., 2022), which are essential for human communication and information exchange. These media files, typically stored and transmitted as byte sequences on electronic devices, enable bGPT to process such digital content for generative modelling and classification.

bGPT trains in generative modelling for representation learning via next token prediction. It then uses features from the final patch-level decoder layer, employing average pooling to derive global features for classification.

To streamline the training process, we standardized audio and image datasets. Audio files were converted to WAV format, with specifications including an 8000 Hz sampling rate, mono channel, and 8-bit depth, each trimmed to one-second lengths. Image data were set to BMP format with a resolution of 32×32, RGB colour, and 24-bit depth.

4.2. Algorithm and Hardware Simulation

To demonstrate the capabilities of byte models in predicting and modelling digital processes, we select two examples—data conversion and CPU state modelling.

Data Conversion: The process involves converting data from one format to another, with symbolic music formats such as ABC notation and MIDI files serving as our main examples. For background information on ABC notation and MIDI, please refer to Appendix A. In this task, bGPT employs the generative modelling approach on concatenated byte sequences of paired ABC and MIDI files, separated by a special patch. The bGPT model learns to convert text-based ABC notation music scores into binary MIDI performance signals and, reversely, convert MIDI back into ABC notation. This necessitates the ability to simulate and reverse-engineer the conversion algorithm[1], which indicates an essential capability for modelling the digital world.

CPU State Modelling: Similarly, the model is fed with

[1] https://github.com/xlvector/abcmidi

Table 1. Overview of datasets for bGPT evaluation, with computational costs benchmarked in NVIDIA V100 GPU hours. Details include file counts in millions, dataset sizes in gigabytes, average file sizes in bytes, and the GPU hours required per epoch for training.

Dataset	Total Files (Million)	Total Bytes (GB)	Mean File Size (Byte)	GPU hours per Epoch	Modality
Wikipedia (Wikimedia, 2022)	6.28	13.67	2237	107	
AG News (Zhang et al., 2015)	0.13	0.03	236.4	2.1	
ImageNet (Deng et al., 2009)	1.33	3.88	3126	18	
CIFAR-10 (Krizhevsky et al., 2009)	0.06	0.17	3126	1.1	
LibriSpeech (Panayotov et al., 2015)	3.68	27.57	8044	84	
Speech Commands v2 (Warden, 2018)	0.09	0.68	8044	2.4	
IrishMAN-ABC (Wu et al., 2023a)	0.21	0.06	313.9	2.4	
IrishMAN-MIDI (Wu et al., 2023a)	0.21	0.39	1916	3.2	
CPU States (proposed)	2.1	6.09	3103	36	

concatenated sequences of low-level machine instructions followed by a series of CPU register states. The objective is to accurately predict how the state updates with each instruction until the program halts. This task demonstrates the capacity of bGPT to interpret operational data and replicate digital activities within hardware.

For CPU state modelling, we introduce the CPU States dataset (with 2.1 million instances), offering a simplified representation of CPU behaviour for ease of data collection and evaluation. Each dataset instance contains a 1KB memory block with varying numbers of machine instructions, followed by a sequence of 16-byte CPU register states. These states include various instructions, totaling 21 unique types with 43 variants, such as data movement, logical operations, and arithmetic operations. Within each state, 1 byte each is allocated for the Program Counter (PC) and Accumulator (ACC), 4 bytes are allocated for the Instruction Register (IR), with an additional 10 bytes reserved for general-purpose registers. Instances are generated by executing random sequences of 1 to 256 instructions and capturing the state after each execution. Despite simplifications, this dataset effectively simulates typical CPU behaviour. See Appendix B for more details.

5. Experiments

5.1. Settings

Our experiments utilized open-source datasets across various domains, including images, speech, text, two types of symbolic music, and CPU states, as detailed in Table 1. The training settings were consistent across all bGPT models and datasets as per Table 2, supporting file sizes up to 8KB using a patch size of 16 and a sequence length of 512. The 110M-parameter bGPT model matches the standard Transformer-based model scale. Notably, we avoided hyperparameter tuning and data augmentation in all evaluations. We employed accuracy (Acc) as the metric to assess classification performance and Bits-Per-Byte (BPB) for other generative modelling tasks, unless otherwise specified.

Table 2. Hyperparameter settings for pre-training and fine-tuning.

Hyperparameter	Pre-training	Fine-tuning
Patch Size	16	16
Patch Length	512	512
Patch-Level Layers	12	12
Byte-Level Layers	3	3
Hidden Size	768	768
Epochs	32	32
Learning Rate	1e-04	1e-05
Batch Size	16	1

5.2. Digital Media Processing

The study aimed to evaluate the effectiveness of bGPT in processing digital media files at the byte level compared to specialized models. We followed the standard pre-training and fine-tuning approach in deep learning, pre-trained bGPT on diverse datasets like ImageNet ($\mathrm{bGPT}_{\mathrm{image}}$), Wikipedia ($\mathrm{bGPT}_{\mathrm{wiki}}$), and LibriSpeech ($\mathrm{bGPT}_{\mathrm{libri}}$). We also explored the impact of mixed pre-training: $\mathrm{bGPT}_{\mathrm{signal}}$ combined the ImageNet and LibriSpeech datasets, while $\mathrm{bGPT}_{\mathrm{mix}}$ integrated all three datasets. A random-initialized variant $\mathrm{bGPT}_{\mathrm{random}}$ was used as a baseline. These models were first fine-tuned using next byte prediction on AG News, CIFAR-10, and Speech Commands v2, and then further fine-tuned for classification.

5.2.1. Baselines

For baseline comparisons, we selected Transformer-based models of similar scale that excel in their respective domains: GPT2-small (Radford et al., 2019) for text generation/classification, and ViT-B/16 (Dosovitskiy et al., 2021) and AST (Gong et al., 2021) for image and audio classification, respectively. GPT2-small was pre-trained on English Wikipedia under the same settings as bGPT. ViT and AST were pre-trained on ImageNet (Deng et al., 2009), and their results were taken from the original studies. In the case of CIFAR-10 (Krizhevsky et al., 2009) and Speech Commands v2 (Warden, 2018), where generative modelling benchmarks were lacking, we did not report BPB metrics.

Table 3. Performance comparison of bGPT models pre-trained on different datasets and baseline models in their respective modalities: GPT2-small on AG News, ViT-B/16 on CIFAR-10, and AST on Speech Commands v2. $\mathrm{bGPT}_{\mathrm{signal}}$ is pre-trained on a merged dataset of ImageNet and LibriSpeech, while $\mathrm{bGPT}_{\mathrm{mix}}$ is pre-trained on a combination of ImageNet, LibriSpeech, and Wikipedia datasets.

	AG News (4 classes)		CIFAR-10 (10 classes)		Speech Commands v2 (36 classes)	
Model	**BPB**	**Acc (%)**	**BPB**	**Acc (%)**	**BPB**	**Acc (%)**
$\mathrm{bGPT}_{\mathrm{random}}$	1.3496	84.74	3.4928	76.73	1.5414	92.43
$\mathrm{bGPT}_{\mathrm{wiki}}$	**1.0639**	**92.49**	3.6663	77.02	1.5719	93.56
$\mathrm{bGPT}_{\mathrm{image}}$	1.4179	83.16	**3.1234**	**88.69**	1.5326	93.91
$\mathrm{bGPT}_{\mathrm{libri}}$	1.3993	83.59	3.3345	83.51	**1.4818**	**96.03**
$\mathrm{bGPT}_{\mathrm{signal}}$	1.4058	83.80	3.1554	87.65	1.4898	95.66
$\mathrm{bGPT}_{\mathrm{mix}}$	1.0935	91.75	3.2279	84.32	1.5086	95.09
Baselines	0.9237	94.50	——	98.13	——	98.11

5.2.2. Results

Table 3 presents the results of various bGPT and baseline models on different benchmarks. The primary insight from this comparison is the significant influence of pre-training on model performance across all tasks. For example, $\mathrm{bGPT}_{\mathrm{wiki}}$ pre-trained on Wikipedia performs well in text-related tasks, while $\mathrm{bGPT}_{\mathrm{libri}}$, pre-trained on LibriSpeech, excels in spoken content tasks compared to other variants. This indicates that bGPT models can achieve better performance in downstream tasks when there is a close match between pre-training and fine-tuning datasets, aligning with standard pre-training principles (Liu et al., 2022).

Despite not having modality-specific prior knowledge, bGPT models still manage to achieve performances that are on par with the baseline models. For instance, $\mathrm{bGPT}_{\mathrm{wiki}}$ achieved a score of 1.0639 BPB and an accuracy of 92.49% on AG News, which, while not quite at the level of GPT2-small, scored 0.9237 BPB and 94.50% accuracy, still demonstrates competitive performance. Likewise, $\mathrm{bGPT}_{\mathrm{libri}}$ reached an accuracy of 96.03% on Speech Commands v2, coming close to the accuracy of AST at 98.11%. However, in the analysis of CIFAR-10, a noticeable difference is seen with $\mathrm{bGPT}_{\mathrm{image}}$ lagging ViT, with an accuracy of 88.69% against 98.13%. This discrepancy in image tasks is likely due to the sequential processing nature of byte models, which struggle to capture the essential two-dimensional spatial relationships within images (Yu et al., 2023). Despite this, simply scaling the model size while retaining this sequential processing approach could still hold promise for achieving state-of-the-art results (Chen et al., 2020).

bGPT models pre-trained on mixed modalities, namely $\mathrm{bGPT}_{\mathrm{signal}}$ and $\mathrm{bGPT}_{\mathrm{mix}}$, yield performances that generally align with the average performance of models pre-trained on individual modalities. For example, $\mathrm{bGPT}_{\mathrm{mix}}$ on the AG News dataset, with a BPB of 1.0935 and accuracy of 91.75%, outperforms other variants but falls short of the performance demonstrated by $\mathrm{bGPT}_{\mathrm{wiki}}$, which is specifically tuned for text data. Similar trends can be observed in the case of $\mathrm{bGPT}_{\mathrm{signal}}$, which typically shows performance levels that lie somewhere between $\mathrm{bGPT}_{\mathrm{image}}$ and $\mathrm{bGPT}_{\mathrm{libri}}$. This illustrates a trade-off in byte models—while mixed-modality pre-training fosters versatility, it may dilute the depth of domain-specific understanding.

Another noteworthy observation from Table 3 is the mixed results in cross-modal fine-tuning on bGPT models, with both positive and negative transfer effects. Positive transfer occurs when models pre-trained on one data type (e.g., $\mathrm{bGPT}_{\mathrm{libri}}$ on LibriSpeech for audio or $\mathrm{bGPT}_{\mathrm{image}}$ on ImageNet for images) are fine-tuned on tasks of another modality, showing non-trivial improvements over random initialization. This suggests shared byte patterns between modalities like audio and images. However, negative transfer is observed when transitioning between text and other modalities, indicating that the benefits of structured pattern learning in pre-training do not universally apply. Text, as a human-created abstraction, exhibits distinct byte-level organizational patterns that differ significantly from audio and visual data, which may explain the negative transfer effects observed in text-related cross-modal fine-tuning.

To further investigate cross-modal knowledge transfer in byte models, we evaluated their performance on the Speech Commands v2 dataset, transformed into 32×32 BMP spectrograms. This process, converting 8KB audio files to 3KB images, inevitably leads to information loss. However, our focus shifts towards exploring the nuances of knowledge transfer dynamics rather than mere performance gains. We selected two pre-trained models for evaluation: $\mathrm{bGPT}_{\mathrm{image}}$ for its data format consistency with spectrograms and $\mathrm{bGPT}_{\mathrm{libri}}$ for its information similarity with spectrograms (both involving speech). These models were employed for conditional generative modelling and classification, following a procedure akin to that in Table 3.

Table 4. Cross-modal knowledge transfer efficacy of bGPT models on spectrogram-based Speech Commands v2.

Model	BPB	Acc (%)
$\mathrm{bGPT}_{\mathrm{random}}$	0.4052	80.74
$\mathrm{bGPT}_{\mathrm{image}}$	**0.3899**	84.66
$\mathrm{bGPT}_{\mathrm{libri}}$	0.3900	**85.26**

The results of Table 4 reveal closely matched BPB between $\mathrm{bGPT}_{\mathrm{image}}$ and $\mathrm{bGPT}_{\mathrm{libri}}$, suggesting that the disparity observed in their CIFAR-10 performance does not extend to this spectrogram-based task. This is because CIFAR-10, comprising natural scenes, shares fewer patterns with spectrograms compared to the commonalities between spectrograms and raw audio. Notably, $\mathrm{bGPT}_{\mathrm{libri}}$, pre-trained on audio, achieves higher accuracy than $\mathrm{bGPT}_{\mathrm{image}}$ on spectrograms with speech content, indicating effective cross-modal transfer and the importance of content alignment over mere data format. This suggests that byte models possess an inherent capability to discern and translate abstract data features and patterns that are independent of the specific format or modality of the input.

Together, these results confirm the versatility and adaptability of bGPT models in processing digital media data and transferring knowledge across modalities, even when data formats differ. This underscores the potential of byte models for integrating shared knowledge from various sources to enhance digital world comprehension.

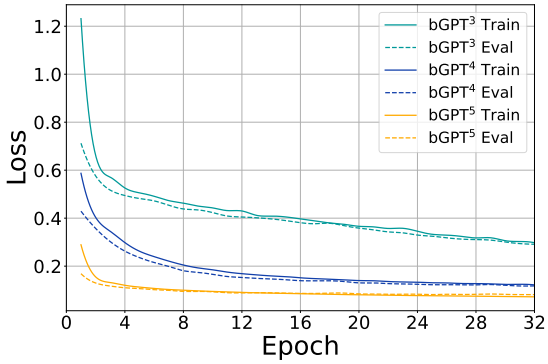
5.3. Algorithm and Hardware Simulation

The primary objective of this evaluation was to assess the capabilities of bGPT in simulating algorithms and hardware. Due to the lack of baseline models and widely used datasets, we investigated its performance across different data scales, evaluating the scalability of bGPT on binary data. This was carried out through two tasks: data conversion and CPU state modelling, utilizing data scales ranging from 10^3 (bGPT^3), 10^4 (bGPT^4), 10^5 (bGPT^5), to 10^6 (bGPT^6). To ensure that the results were primarily driven by data scale, all models were randomly initialized. For data conversion, we utilized the IrishMAN dataset (Wu et al., 2023a), which includes paired ABC notation and MIDI files, evaluated on scales from 10^3 to 10^5. For CPU state modelling, we used the CPU States dataset, evaluated on scales from 10^4 to 10^6.

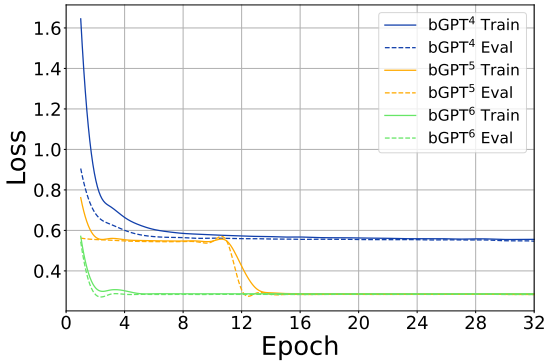
5.3.1. Data Conversion

The task of converting between ABC notation and MIDI evaluates the capability of byte models to simulate algorithms that work with both human-interpretable (ABC notation) and native binary (MIDI) data formats. While previous studies avoided direct MIDI modelling due to its binary nature (Huang et al., 2019; Zeng et al., 2021; Hsiao et al., 2021), byte models are naturally suited for this task, as they can directly work with binary data.

We separately calculated BPB values for each format to clearly measure model performance. For example, in ABC



(a) Data Conversion



(b) CPU State Modelling

Figure 3. Training and evaluation loss curves for different data scales in bGPT performance across epochs.

Table 5. Performance comparison of bGPT across different data scales in data conversion.

	ABC → MIDI		MIDI → ABC	
Model	$\mathrm{BPB}_{\mathrm{abc}}$	$\mathrm{BPB}_{\mathrm{midi}}$	$\mathrm{BPB}_{\mathrm{midi}}$	$\mathrm{BPB}_{\mathrm{abc}}$
bGPT^3	1.4319	0.2381	0.2796	1.1227
bGPT^4	0.9789	0.0202	0.1751	0.1763
bGPT^5	0.7362	0.0011	0.1341	0.0789

Table 6. Performance comparison of bGPT across different data scales in CPU state modelling.

Model	BPB	Acc (%)
bGPT^4	0.6924	27.2404
bGPT^5	0.0969	99.9702
bGPT^6	0.0969	99.9961

to MIDI conversion, $\mathrm{BPB}_{\mathrm{abc}}$ assesses generative modelling, as the model generates content from scratch. While $\mathrm{BPB}_{\mathrm{midi}}$ evaluates data conversion, considering the complete ABC byte sequence is given.

We observe a lower starting loss and a more rapid convergence in Fig. 3a as the data scale grows, indicating that increased data volume directly enhances model performance in simulating the data conversion process. From Table 5, we see that as the scale of data increases, BPB for both ABC to MIDI and MIDI to ABC conversions decrease significantly. The bGPT^5 model achieves an impressively low $\mathrm{BPB}_{\mathrm{midi}}$ of 0.0011 in ABC to MIDI conversion, which is extremely close to perfect performance (where BPB reaches 0), surpassing the performance of the bGPT^3 model by orders of magnitude.

Table 5 suggests consistently higher BPB for ABC in both directions, which is likely due to two factors: 1) The forward conversion from ABC to MIDI focuses on simulating an existing algorithm with necessary information, while the reverse process from MIDI to ABC requires inferring and reconstructing missing information in MIDI files like score structures, musical ornaments, and expressions. 2) As MIDI is a lower-level binary format and ABC notation is a human-readable text format, byte models may find it easier to learn patterns within MIDI files.

5.3.2. CPU State Modelling

CPU state modelling aims to replicate CPU functionality by predicting updates to internal states from machine instructions. We employed bGPT for predicting these states, selecting the highest probability byte at each step based on complete instructions and initial states. The accuracy was assessed through byte-wise comparisons with actual states.

We discovered that data volume significantly influences modelling performance. Table 6 shows significant performance variations, with a notable BPB drop from bGPT^4 to bGPT^5, but diminishing returns beyond bGPT^5. Both bGPT^5 and bGPT^6 achieved near-perfect accuracies (99.97% and 99.99%), suggesting an efficiency beyond simple memorization, given that each test case contained an average of 128 random instructions, and the vast potential combinations of instruction scenarios (over 516 million).

A significant improvement in the performance of bGPT^5 occurred around epoch 11, as shown in Fig. 3b, indicating an emergent ability in CPU state modelling. This leap, especially in BPB and accuracy when comparing bGPT^4 and bGPT^5, suggests a deeper understanding of CPU states may stem from a qualitative enhancement in capability. This aligns with the concept of emergent abilities in large LMs (Wei et al., 2022), where capabilities seem to spontaneously arise with scale and complexity.

However, scepticism exists regarding whether these improvements reflect genuine learning (Schaeffer et al., 2023). Critics argue the performance boosts might be due to non-linear metrics or overfitting. Nonetheless, the linear and smooth nature of BPB counters this, indicating that the improvements likely stem from a real comprehension of CPU operations, suggesting consistent learning rather than metric anomalies.

In summary, bGPT demonstrates strong scalability on native binary data with emergent abilities in data conversion and CPU state modelling, which illuminate its potent capabilities in algorithm and hardware simulation tasks. While the tasks utilized for demonstration in this study were not excessively complicated, the near-perfect performance observed in these contexts points to the broader potential of byte models for simulating and reverse-engineering a wide range of algorithms and hardware.

6. Conclusions

In this paper, we present bGPT as a versatile simulator for the digital world, extending deep learning to binary data processing via next byte prediction. Our experiments demonstrate the effectiveness of bGPT in modelling digital media data, which showcases modality-agnostic knowledge transfer. We observe a strong scalability of bGPT in modelling native binary data and even signs of emergent abilities. bGPT performs comparably to specialized models across diverse datasets without modality-specific designs, and excels in data conversion and CPU state modelling, demonstrating its potential for simulating various algorithms and hardware.

Nonetheless, our experiments illuminate opportunities for improvement. In this study, we confine the modelling to short audio segments and low-resolution images, a consequence of the resource-intensive nature intrinsic to byte models. Due to limited computational resources, we only investigated data conversion between ABC notation and MIDI, without broader assessments across alternate formats. Furthermore, to simplify data collection and evaluation, our CPU state modelling experiments focused solely on simplified CPUs, omitting the use of real modern CPUs, which are considerably more complex.

Future research directions for byte models include: 1) reducing computational cost to make training byte models more feasible; 2) scaling models and dataset sizes to accommodate a broader range of native binary data, as well as handling larger digital media files such as high-resolution images and videos; and 3) improving model performance, particularly for underexplored tasks involving native binary data across diverse application domains.

7. Impact Statements

In this paper, we introduce bGPT, a model designed for binary data processing and digital world modelling, pushing the boundaries of deep learning into the realm of native binary data. This innovation enables bGPT to directly interpret and manipulate binary data, offering profound insights into digital systems. While bGPT presents advancements in understanding and modelling the digital world, it also necessitates a careful examination of its ethical implications and potential impact on societal norms and legal frameworks.

Its ability to simulate or reverse-engineer algorithms and hardware has two major implications: 1) it can significantly boost technological innovation, aiding in the development of cybersecurity, software, and hardware by understanding and improving on existing technologies; 2) it poses a risk to intellectual property, as training bGPT on extensive datasets of paired source code and executable software, might enable the reverse-engineering of proprietary software. This capability, while showcasing its potential, could facilitate unauthorized access to or modification of software, raising security and legal issues.

In conclusion, while bGPT and similar byte models offer exciting opportunities for advancing our understanding and capabilities within the digital world, their deployment requires thoughtful consideration of the ethical, societal, and legal implications. This is especially crucial for safeguarding intellectual property and ensuring security against potential malicious use.

References

Ao, J., Wang, R., Zhou, L., Wang, C., Ren, S., Wu, Y., Liu, S., Ko, T., Li, Q., Zhang, Y., Wei, Z., Qian, Y., Li, J., and Wei, F. Speecht5: Unified-modal encoder-decoder pre-training for spoken language processing. In Muresan, S., Nakov, P., and Villavicencio, A. (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pp. 5723–5738. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.ACL-LONG.393. URL https://doi.org/10.18653/v1/2022.acl-long.393.

Borsos, Z., Marinier, R., Vincent, D., Kharitonov, E., Pietquin, O., Sharifi, M., Roblek, D., Teboul, O., Grangier, D., Tagliasacchi, M., and Zeghidour, N. Audiolm: A language modeling approach to audio generation. *IEEE ACM Trans. Audio Speech Lang. Process.*, 31:2523–2533, 2023. doi: 10.1109/TASLP.2023.3288409. URL https://doi.org/10.1109/TASLP.2023.3288409.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S. M., Nori, H., Palangi, H., Ribeiro, M. T., and Zhang, Y. Sparks of artificial general intelligence: Early experiments with GPT-4. *CoRR*, abs/2303.12712, 2023. doi: 10.48550/ARXIV.2303.12712. URL https://doi.org/10.48550/arXiv.2303.12712.

Chen, M., Radford, A., Child, R., Wu, J., Jun, H., Luan, D., and Sutskever, I. Generative pretraining from pixels. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 1691–1703. PMLR, 2020. URL http://proceedings.mlr.press/v119/chen20s.html.

Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y., and Défossez, A. Simple and controllable music generation. *CoRR*, abs/2306.05284, 2023. doi: 10.48550/ARXIV.2306.05284. URL https://doi.org/10.48550/arXiv.2306.05284.

Défossez, A., Copet, J., Synnaeve, G., and Adi, Y. High fidelity neural audio compression. *CoRR*, abs/2210.13438, 2022. doi: 10.48550/ARXIV.2210.13438. URL https://doi.org/10.48550/arXiv.2210.13438.

Delétang, G., Ruoss, A., Duquenne, P., Catt, E., Genewein, T., Mattern, C., Grau-Moya, J., Wenliang, L. K., Aitchison, M., Orseau, L., Hutter, M., and Veness, J. Language modeling is compression. *CoRR*, abs/2309.10668, 2023. doi: 10.48550/ARXIV.2309.10668. URL https://doi.org/10.48550/arXiv.2309.10668.

Deng, J., Dong, W., Socher, R., Li, L., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*, pp. 248–255. IEEE Computer Society, 2009. doi: 10.1109/CVPR.2009.5206848.

URL https://doi.org/10.1109/CVPR.2009.5206848.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=YicbFdNTTy.

Ferruz, N., Schmidt, S., and Höcker, B. Protgpt2 is a deep unsupervised language model for protein design. *Nature Communications*, 13:4348, 2022. doi: 10.1038/s41467-022-32007-7. URL https://doi.org/10.1038/s41467-022-32007-7.

Gong, Y., Chung, Y., and Glass, J. R. AST: audio spectrogram transformer. In Hermansky, H., Cernocký, H., Burget, L., Lamel, L., Scharenborg, O., and Motlícek, P. (eds.), *Interspeech 2021, 22nd Annual Conference of the International Speech Communication Association, Brno, Czechia, 30 August - 3 September 2021*, pp. 571–575. ISCA, 2021. doi: 10.21437/INTERSPEECH.2021-698. URL https://doi.org/10.21437/Interspeech.2021-698.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *CoRR*, abs/2312.00752, 2023. doi: 10.48550/ARXIV.2312.00752. URL https://doi.org/10.48550/arXiv.2312.00752.

Guo, W., Mu, D., Xing, X., Du, M., and Song, D. DEEPVSA: facilitating value-set analysis with deep learning for postmortem program analysis. In Heninger, N. and Traynor, P. (eds.), *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, pp. 1787–1804. USENIX Association, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/guo.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, 1997. doi: 10.1162/NECO.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. Training compute-optimal large language models. *CoRR*, abs/2203.15556, 2022. doi: 10.48550/ARXIV.2203.15556. URL https://doi.org/10.48550/arXiv.2203.15556.

Horton, M., Mehta, S., Farhadi, A., and Rastegari, M. Bytes are all you need: Transformers operating directly on file bytes. *CoRR*, abs/2306.00238, 2023. doi: 10.48550/ARXIV.2306.00238. URL https://doi.org/10.48550/arXiv.2306.00238.

Hsiao, W., Liu, J., Yeh, Y., and Yang, Y. Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pp. 178–186. AAAI Press, 2021. doi: 10.1609/AAAI.V35I1.16091. URL https://doi.org/10.1609/aaai.v35i1.16091.

Huang, C. A., Vaswani, A., Uszkoreit, J., Simon, I., Hawthorne, C., Shazeer, N., Dai, A. M., Hoffman, M. D., Dinculescu, M., and Eck, D. Music transformer: Generating music with long-term structure. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL https://openreview.net/forum?id=rJe4ShAcF7.

Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. 2009.

Kudo, T. Subword regularization: Improving neural network translation models with multiple subword candidates. In Gurevych, I. and Miyao, Y. (eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 1: Long Papers*, pp. 66–75. Association for Computational Linguistics, 2018. doi: 10.18653/V1/P18-1007. URL https://aclanthology.org/P18-1007/.

Kudo, T. and Richardson, J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Blanco, E. and Lu, W. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018: System Demonstrations, Brussels,*

Belgium, October 31 - November 4, 2018, pp. 66–71. Association for Computational Linguistics, 2018. doi: 10.18653/V1/D18-2012. URL https://doi.org/10.18653/v1/d18-2012.

LeCun, Y., Boser, B. E., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W. E., and Jackel, L. D. Handwritten digit recognition with a back-propagation network. In Touretzky, D. S. (ed.), *Advances in Neural Information Processing Systems 2, [NIPS Conference, Denver, Colorado, USA, November 27-30, 1989]*, pp. 396–404. Morgan Kaufmann, 1989. URL http://papers.nips.cc/paper/293-handwritten-digit-recognition-with-a-back-propagation-network.

Li, J., Li, D., Savarese, S., and Hoi, S. C. H. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19730–19742. PMLR, 2023. URL https://proceedings.mlr.press/v202/li23q.html.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. *CoRR*, abs/2304.08485, 2023. doi: 10.48550/ARXIV.2304.08485. URL https://doi.org/10.48550/arXiv.2304.08485.

Liu, Z., Xu, Y., Xu, Y., Qian, Q., Li, H., Ji, X., Chan, A. B., and Jin, R. Improved fine-tuning by better leveraging pre-training data. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/d1c88f9790765146ec8fb5d02e5653a0-Abstract-Conference.html.

Nguyen, E., Poli, M., Faizi, M., Thomas, A. W., Birch-Sykes, C., Wornow, M., Patel, A., Rabideau, C. M., Massaroli, S., Bengio, Y., Ermon, S., Baccus, S. A., and Ré, C. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution. *CoRR*, abs/2306.15794, 2023. doi: 10.48550/ARXIV.2306.15794. URL https://doi.org/10.48550/arXiv.2306.15794.

Notin, P., Dias, M., Frazer, J., Marchena-Hurtado, J., Gomez, A. N., Marks, D. S., and Gal, Y. Tranception: Protein fitness prediction with autoregressive transformers and inference-time retrieval. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 16990–17017. PMLR, 2022. URL https://proceedings.mlr.press/v162/notin22a.html.

Oord, A. v. d., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., and Kavukcuoglu, K. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

Panayotov, V., Chen, G., Povey, D., and Khudanpur, S. Librispeech: An ASR corpus based on public domain audio books. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2015, South Brisbane, Queensland, Australia, April 19-24, 2015*, pp. 5206–5210. IEEE, 2015. doi: 10.1109/ICASSP.2015.7178964. URL https://doi.org/10.1109/ICASSP.2015.7178964.

Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. Deep contextualized word representations. In Walker, M. A., Ji, H., and Stent, A. (eds.), *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, pp. 2227–2237. Association for Computational Linguistics, 2018. doi: 10.18653/V1/N18-1202. URL https://doi.org/10.18653/v1/n18-1202.

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., and Nicholas, C. K. Malware detection by eating a whole EXE. In *The Workshops of the The Thirty-Second AAAI Conference on Artificial Intelligence, New Orleans, Louisiana, USA, February 2-7, 2018*, volume WS-18 of *AAAI Technical Report*, pp. 268–276. AAAI Press, 2018. URL https://aaai.org/ocs/index.php/WS/AAAIW18/paper/view/16422.

Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., and Chen, M. Hierarchical text-conditional image generation with CLIP latents. *CoRR*, abs/2204.06125, 2022. doi: 10.48550/ARXIV.2204.06125. URL https://doi.org/10.48550/arXiv.2204.06125.

Rubenstein, P. K., Asawaroengchai, C., Nguyen, D. D., Bapna, A., Borsos, Z., de Chaumont Quitry, F., Chen, P., Badawy, D. E., Han, W., Kharitonov, E., Muckenhirn, H., Padfield, D., Qin, J., Rozenberg, D., Sainath, T. N., Schalkwyk, J., Sharifi, M., Ramanovich, M. T., Tagliasacchi, M., Tudor, A., Velimirovic, M., Vincent, D., Yu, J., Wang, Y., Zayats, V., Zeghidour, N., Zhang, Y., Zhang, Z., Zilka, L., and Frank, C. H. Audiopalm: A large language model that can speak and listen. *CoRR*, abs/2306.12925, 2023. doi: 10.48550/ARXIV.2306.12925. URL https://doi.org/10.48550/arXiv.2306.12925.

Schaeffer, R., Miranda, B., and Koyejo, S. Are emergent abilities of large language models a mirage? In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=ITw9edRDlD.

Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics, 2016. doi: 10.18653/V1/P16-1162. URL https://doi.org/10.18653/v1/p16-1162.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023a. doi: 10.48550/ARXIV.2302.13971. URL https://doi.org/10.48550/arXiv.2302.13971.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023b. doi: 10.48550/ARXIV.2307.09288. URL https://doi.org/10.48550/arXiv.2307.09288.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wang, C., Cho, K., and Gu, J. Neural machine translation with byte-level subwords. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pp. 9154–9160. AAAI Press, 2020. doi: 10.1609/AAAI.V34I05.6451. URL https://doi.org/10.1609/aaai.v34i05.6451.

Wang, C., Chen, S., Wu, Y., Zhang, Z., Zhou, L., Liu, S., Chen, Z., Liu, Y., Wang, H., Li, J., He, L., Zhao, S., and Wei, F. Neural codec language models are zero-shot text to speech synthesizers. *CoRR*, abs/2301.02111, 2023. doi: 10.48550/ARXIV.2301.02111. URL https://doi.org/10.48550/arXiv.2301.02111.

Wang, J., Gangavarapu, T., Yan, J. N., and Rush, A. M. Mambabyte: Token-free selective state space model, 2024.

Warden, P. Speech commands: A dataset for limited-vocabulary speech recognition. *CoRR*, abs/1804.03209, 2018. URL http://arxiv.org/abs/1804.03209.

Wei, J., Liu, Q., Guo, Y., and Jiang, X. Training multilingual pre-trained language model with byte-level subwords. *arXiv preprint arXiv:2101.09469*, 2021.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., and Fedus, W. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, 2022, 2022. URL https://openreview.net/forum?id=yzkSU5zdwD.

Wikimedia. Wikimedia downloads, 2022. URL https://dumps.wikimedia.org.

Wu, S., Li, X., Yu, F., and Sun, M. Tunesformer: Forming irish tunes with control codes by bar patching. In Porcaro, L., Batlle-Roca, R., and Gómez, E. (eds.), *Proceedings of the 2nd Workshop on Human-Centric Music Information Retrieval 2023 co-located with the 24th International Society for Music Information Retrieval Conference (ISMIR 2023), Milan, Italy, November 10, 2023*, volume 3528 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2023a. URL https://ceur-ws.org/Vol-3528/paper1.pdf.

Wu, S., Yu, D., Tan, X., and Sun, M. Clamp: Contrastive language-music pre-training for cross-modal symbolic music information retrieval. In Sarti, A., Antonacci, F., Sandler, M., Bestagini, P., Dixon, S., Liang, B., Richard, G., and Pauwels, J. (eds.), *Proceedings of the 24th International Society for Music Information Retrieval Conference, ISMIR 2023, Milan, Italy, November 5-9, 2023*, pp. 157–165, 2023b. doi: 10.5281/ZENODO.10265247. URL https://doi.org/10.5281/zenodo.10265247.

Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Kaiser, L., Gouws, S., Kato, Y., Kudo, T., Kazawa, H., Stevens, K., Kurian, G., Patil, N., Wang, W., Young, C., Smith, J., Riesa, J., Rudnick, A., Vinyals, O., Corrado, G., Hughes, M., and Dean, J. Google's neural machine translation system: Bridging the gap between human and machine translation. *CoRR*, abs/1609.08144, 2016. URL http://arxiv.org/abs/1609.08144.

Xue, L., Barua, A., Constant, N., Al-Rfou, R., Narang, S., Kale, M., Roberts, A., and Raffel, C. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Trans. Assoc. Comput. Linguistics*, 10:291–306, 2022. doi: 10.1162/TACL_A_00461. URL https://doi.org/10.1162/tacl_a_00461.

Yang, D., Tian, J., Tan, X., Huang, R., Liu, S., Chang, X., Shi, J., Zhao, S., Bian, J., Wu, X., et al. Uniaudio: An audio foundation model toward universal audio generation. *arXiv preprint arXiv:2310.00704*, 2023.

Yu, L., Simig, D., Flaherty, C., Aghajanyan, A., Zettlemoyer, L., and Lewis, M. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=JTmO2V9Xpz.

Zeng, M., Tan, X., Wang, R., Ju, Z., Qin, T., and Liu, T. Musicbert: Symbolic music understanding with large-scale pre-training. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, volume ACL/IJCNLP 2021 of *Findings of ACL*, pp. 791–800. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.FINDINGS-ACL.70. URL https://doi.org/10.18653/v1/2021.findings-acl.70.

Zhang, X., Zhao, J. J., and LeCun, Y. Character-level convolutional networks for text classification. In Cortes, C., Lawrence, N. D., Lee, D. D., Sugiyama, M., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, pp. 649–657, 2015. URL https://proceedings.neurips.cc/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html.

Zhu, D., Chen, J., Shen, X., Li, X., and Elhoseiny, M. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *CoRR*, abs/2304.10592, 2023. doi: 10.48550/ARXIV.2304.10592. URL https://doi.org/10.48550/arXiv.2304.10592.

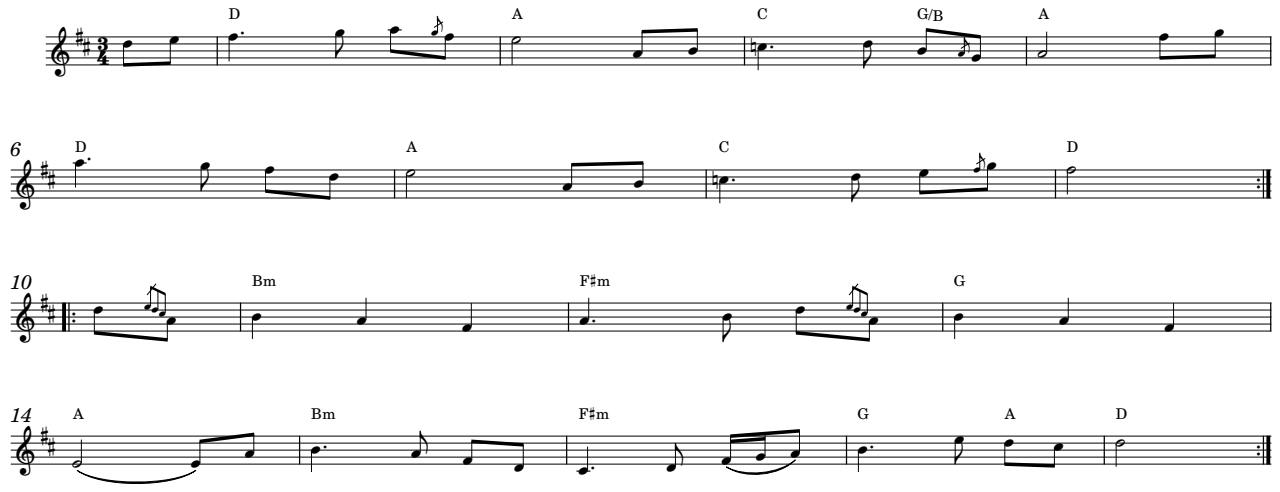


Figure 4. Sheet music example corresponding to the ABC notation of a folk tune in D major.

A. Symbolic Music Formats

A.1. ABC Notation

ABC notation is a shorthand form of music notation that uses ASCII characters to represent musical notes and symbols. It is a compact, human-readable, and easily typed format that has gained popularity for its ability to facilitate the swift digital transcription, sharing, and editing of music. Originating within folk and traditional music circles, ABC notation has become a universal method for musicians and enthusiasts to exchange music without the need for proprietary software or extensive musical training.

To illustrate the fundamentals of ABC notation, consider the example below, which depicts a folk tune:

```
X:1
L:1/8
M:3/4
K:D
de |"D" f3 g a{/g}f |"A" e4 AB |"C" =c3 d"G/B" B{/A}G |"A" A4 fg |
"D" a3 g fd |"A" e4 AB |"C" =c3 d e{/f}g |"D" f4 ::
d{/edc}A |"Bm" B2 A2 F2 |"F#m" A3 B d{/edc}A |"G" B2 A2 F2 |
"A" (E4 E)A |"Bm" B3 A FD |"F#m" C3 D (F/G/A) |"G" B3 e"A" dc |"D" d4 :|
```

The corresponding sheet music rendered from this ABC sequence is shown in Fig. 4. In this example, the structure of ABC notation starts with a header that includes:

- **X (Tune Number)**: This is a unique number assigned to the tune, serving as a reference in collections or databases.
- **L (Unit Note Length)**: This defines the default duration of notes. In this context, `1/8` indicates that each note is an eighth note, which is a standard choice for folk tunes.
- **M (Meter)**: This denotes the time signature of the piece, which determines the rhythmic pulse. Here, `3/4` suggests a waltz-like rhythm.
- **K (Key)**: This sets the key signature for the tune. `D` signifies that the piece is in the key of D major.

After the header, the body of the ABC notation consists of the musical notes themselves. Each note is represented by a letter (`A` to `G`), which corresponds to the pitch. The octaves are indicated by lower-cased letters or apostrophes for higher octaves and upper-cased letters or commas for lower or higher octaves, respectively. The duration of notes is modified by numbers; for instance, a `2` following a note would double its value, while a `/2` would halve it. Chords can be notated by enclosing the

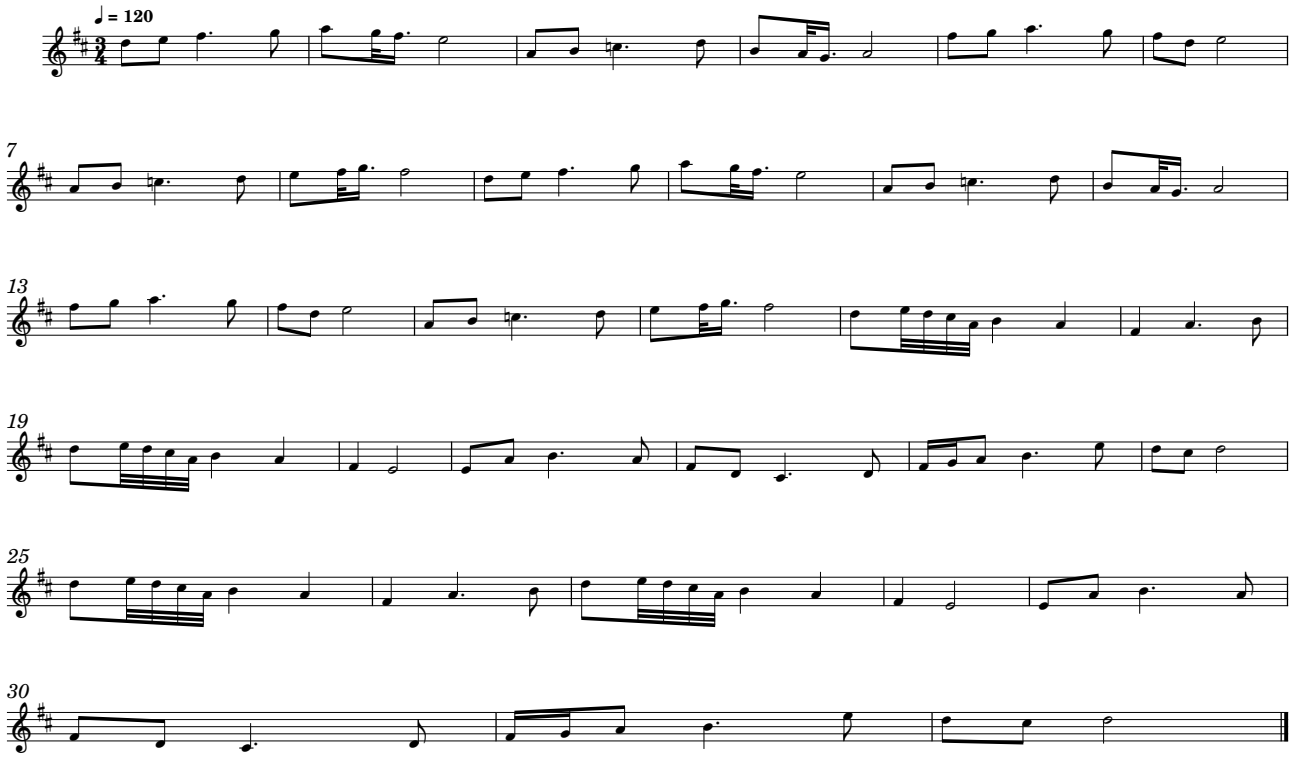


Figure 5. Sheet music example corresponding to the MIDI of the same folk tune presented in Fig. 4.

chord name in quotation marks to suggest harmony, such as `"F#m"` representing the F sharp minor. Additional symbols are used for sharps ^, flats _, and naturals = to alter pitch, and various other characters represent articulations and expressions.

This notation describes the tune with sufficient information, from its rhythmic structure down to the details of its melody and harmony. As such, ABC notation serves as a bridge between the traditional form of musical scores and the modern digital world, offering a text-based alternative that is both efficient and widely accessible. For a detailed documentation of ABC notation, please refer to the ABC notation standard[2].

A.2. MIDI

MIDI, short for Musical Instrument Digital Interface, is a technical standard that describes a protocol, digital interface, and connectors and allows a wide variety of electronic musical instruments, computers, and other related devices to connect and communicate with one another. Unlike ABC notation, which is designed for human readability and the transcription of musical scores, MIDI is focused on capturing and conveying information about music performance and production in a format that machines can interpret and process.

The core of MIDI communication involves MIDI messages, which are digital signals that represent various musical elements such as notes, pitch, velocity (how hard a note is played), control changes (modifications to sound parameters), and program changes (instrument changes). These messages do not transmit audio; instead, they send instructions about how music should be played or produced. This approach allows MIDI to be incredibly versatile and lightweight compared to raw audio, suitable for real-time control during live performances and detailed editing in studio environments.

MIDI files, commonly with the extension .mid or .midi, store sequences of MIDI messages that can be played back by computer software or electronic instruments. These files make it possible to share and distribute musical compositions in a form that can be easily modified, transposed, or played back with different instrument sounds, offering a level of flexibility and creative freedom that is unique to the digital music domain.

[2] https://abcnotation.com/wiki/abc:standard:v2.1

There are several ways to create and edit MIDI, each catering to different workflows and preferences:

- **MIDI Keyboards and Controllers**: Physical devices that allow musicians to input notes and control data into a computer. They emulate traditional instruments and can capture the nuances of a performance in MIDI format.
- **Piano Roll**: A graphical interface in digital audio workstations (as shown in Fig. 1) where notes are displayed and edited on a grid representing pitch and time. It is intuitive for composing melodies, chords, and rhythms.
- **Score Editor**: Allows users to input and edit music in traditional notation, converting written notes into MIDI data. Ideal for those familiar with musical notation.
- **Programming and Scripting**: Involves writing code to generate and manipulate MIDI data, offering complex and generative musical possibilities beyond manual input methods. For instance, complex MIDI sequences and automation can be generated using software such as Max/MSP, Pure Data, or programming languages (e.g., SuperCollider).

Although MIDI offers immense benefits for musicians and producers, when converting MIDI data back into score-oriented symbolic music formats like ABC notation, certain challenges arise due to the intrinsic differences between these formats. MIDI excels at capturing how a piece of music is played, including the timing, velocity, and expressiveness of each note. In contrast, ABC notation focuses on the structural elements of music, such as melody, harmony, and rhythm, using a textual format that is more accessible for human reading and writing but less detailed in terms of performance nuances.

To illustrate the conversion between MIDI and ABC notation, consider the following example:

```
X:1
L:1/8
M:3/4
K:D
de f3 g | ag/<f/ e4 | AB =c3 d | BA/<G/ A4 | fg a3 g | fd e4 |
AB =c3 d | ef/<g/ f4 | de f3 g | ag/<f/ e4 | AB =c3 d | BA/<G/ A4 |
fg a3 g | fd e4 | AB =c3 d | ef/<g/ f4 | de/4d/4c/4A/4 B2 A2 | F2 A3 B |
de/4d/4c/4A/4 B2 A2 | F2 E4 | EA B3 A | FD C3 D | F/G/A B3 e | dc d4 |
de/4d/4c/4A/4 B2 A2 | F2 A3 B | de/4d/4c/4A/4 B2 A2 | F2 E4 | EA B3 A |
FD C3 D | F/G/A B3 e | dc d4 |]
```

This is the same tune we have demonstrated in Fig, 4, but it has now been converted from ABC notation to MIDI and subsequently back to ABC notation. The corresponding sheet music, rendered from this ABC sequence, is shown in Fig. 5. One can observe that the ABC sequence and the musical score appear significantly longer than the original due to the elimination of repeat signs in MIDI, requiring repeated sections to be explicitly written out. Additionally, the bar divisions and the ornaments in the MIDI-to-ABC conversion might not align perfectly with the original ABC notation. These ornaments are often represented as a series of rapid notes in MIDI, which can complicate their translation back into ABC notation.

While MIDI-rendered sheet music and its ABC notation counterpart may sound identical when performed, the visual complexity of the MIDI transcription can make it more challenging to read than the ABC version. This discrepancy underscores the fact that MIDI and ABC are formats tailored for different contexts. MIDI is adept at capturing nuanced performance data, such as dynamics and subtle timing variations, which are essential for playback fidelity but are not as visually straightforward as ABC notation. On the other hand, ABC notation, with its simplified symbolic representation, is not ideal for conveying detailed performance information.

Both formats have overlapping capabilities in representing musical information, yet each also possesses unique characteristics that do not translate perfectly when converting from one to the other. This results in a loss of information: nuances specific to each format can be diminished or omitted entirely during the conversion process. Such losses present a challenge in preserving the full intent and detail of a musical piece when moving between ABC and MIDI formats.

B. CPU States Dataset

The CPU States Dataset has been crafted using a Python script to simulate the operations of a CPU, offering a controlled environment to assess the capabilities of bGPT in modelling the hardware behaviour. This appendix offers an overview of the dataset, introducing the structure, instruction set, and details of the dataset curation.

The dataset comprises entries of 1KB memory blocks, each paired with a subsequent 16-byte sequence representing the CPU register states. The memory blocks encode varying numbers of machine instructions, while the register state sequence includes the CPU status post-execution of each instruction. The CPU registers in each state consist of:

- **Program Counter (PC)**: 1 byte in size, points to the next instruction in the memory.
- **Accumulator (ACC)**: A 1-byte register involved in arithmetic and logic operations.
- **Instruction Register (IR)**: Occupies 4 bytes, storing the currently executed instruction.
- **General-Purpose Registers**: An additional 10 bytes are allocated, emulating a range of registers (labelled A-J) used for various computational purposes.

Transitions between states mimic the execution cycle of a CPU, including fetching instructions from memory, decoding them to determine the required operation, and executing the operation to modify the state. The execution process includes instructions for modifying register values, performing arithmetic calculations, and controlling the program flow.

Each instruction in the dataset is encoded in a four-byte format `OP ADDR1 ADDR2 ADDR3`, with each byte serving a specific purpose:

- **OP (Operation Code)**: 1 byte representing the opcode of the instruction to be executed.
- **ADDR1, ADDR2, ADDR3**: 1 byte each, representing up to three addresses that the instruction may interact with.

The opcode and addresses together describe the action to be taken by the CPU. The instructions vary in terms of the number of addresses they use, from zero-address to three-address instructions. This variability allows the simulation of a wide range of operations, from simple data movement to complex arithmetic and logical functions.

Table 7 (located on page 19) details the instruction set and the variants each instruction can have, depending on the values in the address fields. The variant to be executed is determined by the non-zero value in the address fields. For zero-address instructions, all address fields are zero. For single-address instructions, ADDR1 holds the address of the register involved, and the other fields are zero. For two- and three-address instructions, the respective fields hold the addresses of the registers involved in the operation.

In designing the instruction set, complex instructions like jump commands were deliberately excluded to prevent the occurrence of infinite loops or other complexities that could hinder straightforward simulation.

The Python script generates the CPU States Dataset by randomly initializing the ACC and general-purpose registers for each simulation, thereby ensuring a wide range of starting conditions. It then executes a random sequence of instructions, varying in length from 1 to 256, with the effect of each instruction on the CPU state being recorded. This results in a sequence of states that reflects the cumulative impact of the executed instructions. The script guarantees that each sequence culminates with a halt instruction `HLT`, marking the end of a program. Furthermore, when considering only the register changes related to instructions, there are over 516 million potential scenarios resulting from combinations of various instruction types, registers, and their values.

The dataset thus produced comprises 2.1 million entries for the training set and 21,000 entries for the evaluation set. This process, with its random initial states and varied instruction lengths, produces a diverse dataset that is essential for training and evaluating the effectiveness of bGPT in modelling CPU states.

To aid in a more intuitive comprehension of the operational logic of the CPU, a randomly generated program for executing 10 instructions along with their corresponding states in human-readable form is provided below:

```
Program: ['MUL J A', 'DIV I', 'MUL E D', 'ADD C', 'LOADI 86', 'MOV A H', 'AND D E', 'POP H', 'CLR', 'HLT']

State at step 0:
PC: 0
ACC: 146
IR: HLT
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 174, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 2}
```

```
State at step 1:
PC: 1
ACC: 146
IR: MUL J A
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 174, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 2}

State at step 2:
PC: 2
ACC: 146
IR: DIV I
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 174, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 3:
PC: 3
ACC: 1
IR: MUL E D
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 174, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 4:
PC: 4
ACC: 1
IR: ADD C
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 5:
PC: 5
ACC: 246
IR: LOADI 86
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 6:
PC: 6
ACC: 86
IR: MOV A H
Registers: {'A': 19, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 7:
PC: 7
ACC: 86
IR: AND D E
Registers: {'A': 20, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 8:
PC: 8
ACC: 86
IR: POP H
Registers: {'A': 20, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 20, 'I': 140, 'J': 38}

State at step 9:
PC: 9
ACC: 86
IR: CLR
Registers: {'A': 20, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 86, 'I': 140, 'J': 38}

State at step 10:
PC: 10
ACC: 0
IR: HLT
Registers: {'A': 20, 'B': 55, 'C': 245, 'D': 35, 'E': 255, 'F': 185, 'G': 9, 'H': 86, 'I': 140, 'J': 38}
```

The dataset is intended for use in machine learning research, particularly for developing models capable of predicting CPU behaviour. It can also serve as an educational tool, helping students understand CPU operation principles.

Table 7. Defined instruction set and the address variants of different instructions.

Instruction	Address Variants	Meaning
HLT	0	Halts the CPU with “HLT”
CLR	0, 1	Zeroes the accumulator with “CLR” or a specific register with “CLR A”
INC	0, 1	Increments the accumulator with “INC” or a specific register with “INC A”
DEC	0, 1	Decrements the accumulator with “DEC” or a specific register with “DEC A”
SHL	0, 1	Shifts the accumulator left by one bit with “SHL” or a specific register with “SHL A”
SHR	0, 1	Shifts the accumulator right by one bit with “SHR” or a specific register with “SHR A”
ROL	0, 1	Rotates the accumulator left by one bit with “ROL” or a specific register with “ROL A”
ROR	0, 1	Rotates the accumulator right by one bit with “ROR” or a specific register with “ROR A”
NOT	0, 1	Performs bitwise NOT on the accumulator with “NOT” or a specific register with “NOT A”
PUSH	1	Pushes the value of a specific register onto the stack with “PUSH A”
POP	1	Pops the top of the stack into a specific register with “POP A”
LOADI	1	Loads an immediate value into the accumulator with “LOADI value”
SWAP	1, 2	Swaps values between the accumulator and a register with “SWAP A” or between two registers with “SWAP A B”
ADD	1, 2, 3	Adds using one, two, or three addresses with examples “ADD A”, “ADD A B”, “ADD A B C”
SUB	1, 2, 3	Subtracts using one, two, or three addresses with examples “SUB A”, “SUB A B”, “SUB A B C”
MUL	1, 2, 3	Multiplies using one, two, or three addresses with examples “MUL A”, “MUL A B”, “MUL A B C”
DIV	1, 2, 3	Divides using one, two, or three addresses with examples “DIV A”, “DIV A B”, “DIV A B C”
AND	1, 2, 3	Performs bitwise AND using one, two, or three addresses with examples “AND A”, “AND A B”, “AND A B C”
OR	1, 2, 3	Performs bitwise OR using one, two, or three addresses with examples “OR A”, “OR A B”, “OR A B C”
XOR	1, 2, 3	Performs bitwise XOR using one, two, or three addresses with examples “XOR A”, “XOR A B”, “XOR A B C”
MOV	2	Moves a value from one register to another with “MOV A B”